U4: Trijų Taisyklė (Kopijavimo elgsena) ir Operatorių Perkrovimas

Savaitės: 7-8
Svoris: 1 balas (+ BONUS: 0,2 balai)
Terminas: Savaitės 8 pabaiga

🎯 Užduoties tikslas

Išmokti taikyti Trijų Taisyklės (Rule of Three) principą klasėms, kurios valdo dinaminę atmintį.

📚 Mokymosi tikslai

Atlikę šią užduotį, mokėsite:

- ☑ Suprasti **paviršinio** (shallow) vs **giliojo** (deep) kopijavimo skirtumus
- ☑ Implementuoti **kopijavimo konstruktorių** (copy constructor) su dinamine atmintimi
- ☑ Implementuoti **kopijavimo priskyrimo operatorių** (copy assignment operator)
- ☑ Taikyti **Trijų Taisyklės** (Rule of Three) principą
- ☑ Dirbti su C stiliaus eilutėmis (C-style strings) `char*` C++ priemonėmis
- ☑ Perkrauti operatorius (operator overloading)

📝 Pastaba apie komentarus kode

Šioje užduotyje naudosime `// TODO:` komentarus, kurie žymi užduotis, kurias jums reikia atlikti.

TODO (angl. "to do" — "padaryti") yra **tarptautinė programavimo praktika** — tokius komentarus rasite bet kuriame profesionaliame projekte. IDE (pvz., VS Code, CLion) net specialiai paryškina `TODO` komentarus, kad lengviau juos pastebėtumėme.

💡 Kas yra Trijų Taisyklė (Rule of Three)?

U3 užduotyje susidūrėme su **kopijavimo konstruktoriaus** (copy constructor) problema naudodami `vector<Window>`. Matėme, kad objektai kopijuojami automatiškai, ir reikėjo vartotojo apibrėžto (user-defined) kopijavimo konstruktoriaus (kad pasiaiškinti programos elseną, jos "virtuvę").

U4 užduotyje **grįžtame prie pagrindų** — "pamiršime" `vector`, `Room`, kompoziciją. Naudosime **paprastas klases su dinamine atmintimi** (`IntArray`), kad suprastume, **kaip teisingai kopijuoti** objektus su dinamine atmintimi (t.y. objektus tokių klasių, kurios numato dinaminės atminties panaudojimą).

Trijų Taisyklė (Rule of Three) — C++ taisyklė: jei klasei reikia **vieno** iš šių trijų, reikia **visų trijų**:

1. **Destruktorius** (destructor: `~ClassName()`)
2. **Kopijavimo konstruktorius** (copy constructor: `ClassName(const ClassName&)`)

3. **Kopijavimo priskyrimo operatorius** (copy assignment operator: `operator=`)

Kodėl? Kai klasė valdo **dinaminę atmintį**, numatytasis (default) kopijavimas įtakoja **dvigubą (atminties) atlaisvinimą** (double delete) ir (paprastai) sukelia **programos griūtį/lūžį** (crash).

📋 Užduoties struktūra

DALIS 1: IntArray (žingsniai 1-3)

- Problema → Kopijavimo konstruktorius → Trijų Taisyklė

DALIS 2: MyString (žingsniai 4-6)

- Trijų Taisyklė → Sujungimo operatorius → Kiti operatoriai (BONUS)

DALIS 1: IntArray

Tikslas: Suprasti Trijų Taisyklę (Rule of Three) **izoliuotai**, be kompozicijos, be `vector`.

Kodėl IntArray? Paprasta klasė su `int*` ir dinamine atmintimi — idealus pavyzdys demonstruoti paviršinį (shallow) vs gilųjį (deep) kopijavimą. Primityvūs duomenys (`int`), paprasta logika, aiškiai matoma problema.

Po IntArray pereisime prie labiau **realaus** *use case* — `MyString` su `char*`.

---

1 žingsnis: Problema - Paviršinis kopijavimas... ir griūtis

Direktorija: `U4/01/`

Motyvacija:

U3 BONUS matėme, kad objektai **automatiškai kopijuojami** (`vector.push_back()`).

U4 **grįžtame prie pagrindų** — pamatysime **paprastą** pavyzdį su dinamine atmintimi, kad suprastume **kodėl** ir **kaip** teisingai kopijuoti objektus.

Reikalavimai:

Sukurkite `IntArray` klasę su dinamine atmintimi.

IntArray.h:

```
#ifndef INTARRAY_H
#define INTARRAY_H

#include <iostream>

class IntArray {
private:
    int* data;
    int size;

public:
    IntArray(int s);
    ~IntArray();

    // Getters
    int getSize() const;
    int get(int index) const;
    void set(int index, int value);

    void print() const;
};

#endif
```

IntArray.cpp

```
#include "IntArray.h"

IntArray::IntArray(int s) : size(s) {
    // TODO: Sukurti dinamini masyvą, inicializuoti nuliukais, logging
    // data = new int[size];
    // for (int i = 0; i < size; i++) {
    //     data[i] = 0;
    // }
    // std::cout << "[CTOR] IntArray created (size=" << size
    //           << ", data=" << data << ")" << std::endl;
}

IntArray::~IntArray() {
    // TODO: Logging, atlaisvinti atmintį
    // std::cout << "[DTOR] IntArray destroyed (size=" << size
    //           << ", data=" << data << ")" << std::endl;
    // delete[] data;
}

int IntArray::getSize() const {
    // TODO: return size;
}

int IntArray::get(int index) const {
    // TODO: ribų patikrinimas; return data[index]
    // if (index < 0 || index >= size) {
    //    std::cerr << "Index out of bounds!" << std::endl;
    //    return 0;
    // }
    // return data[index];
}

void IntArray::set(int index, int value) {
    // TODO: ribų patikrinimas; ...
}

void IntArray::print() const {
    // TODO: Išspausdinti masyvą
    // std::cout << "IntArray[" << size << "]: ";
    // for (int i = 0; i < size; i++) {
    //     std::cout << data[i] << " ";
    // }
    // std::cout << std::endl;
}
```

Testavimas:

main.cpp:

```
#include <iostream>
#include "IntArray.h"

int main() {
    std::cout << "=== Paviršinio kopijavimo problema ===" << std::endl;

    IntArray arr1(5);
    arr1.set(0, 10);
    arr1.set(1, 20);
    arr1.set(2, 30);

    std::cout << "arr1: ";
    arr1.print();

    IntArray arr2 = arr1;  // ← shallow copy!

    std::cout << "arr2: ";
    arr2.print();

    arr2.set(0, 999);

    std::cout << "arr2 (po keitimo): ";
    arr2.print();

    std::cout << "arr1: ";
    arr1.print();  // ← arr1 IRGI pasikeitė!

    std::cout << "Programa baigiasi..." << std::endl;

    return 0;
}
```

Galima išvestis:

```
=== Paviršinio kopijavimo problema ===
[CTOR] IntArray created (size=5, data=0x13b41b21860)
arr1: IntArray[5]: 10 20 30 0 0
arr2: IntArray[5]: 10 20 30 0 0
arr2 (po keitimo): IntArray[5]: 999 20 30 0 0
arr1: IntArray[5]: 999 20 30 0 0                        ← Kodėl? BLOGAI!
Programa baigiasi...
[DTOR] IntArray destroyed (size=5, data=0x13b41b21860)  ← Tas pats adresas!
[DTOR] IntArray destroyed (size=5, data=0x13b41b21860)  ← Tas pats adresas!
❌
```

Svarbu!

Paviršinio kopijavimo (shallow copy) problema:

- Kompiliatorius generuoja numatytąjį (default) kopijavimo konstruktorių
- Kopijuoja **rodyklę**, ne **duomenis** į kuriuos ji rodo!
- `arr1.data == arr2.data` → tas pats adresas
- Destruktorius: `delete[] data` kviečiamas **du kartus** → **dvigubas atlaisvinimas** (double delete) → **CRASH!**

Failų struktūra:

```
U4/01/
├── IntArray.h
├── IntArray.cpp
├── main.cpp
└── Makefile
```

---

2 žingsnis: Kopijavimo konstruktorius - Gilusis kopijavimas

Direktorija: `U4/02/`

Motyvacija:

Matėme problemą. Spręsime ją — **gilusis kopijavimas** (deep copy) su kopijavimo konstruktoriumi!

Reikalavimai:

Pridėkite **kopijavimo konstruktorių** (copy constructor) į `IntArray` klasę.

IntArray.h (papildyti):

```
class IntArray {
    // ... ankstesnis kodas ...
public:
    IntArray(int s);
    ~IntArray();

    // NAUJAS: Kopijavimo konstruktorius
    IntArray(const IntArray& other);  // ← Pridėti

    // ... kiti metodai ...
};
```

IntArray.cpp (papildyti):

```
// Kopijavimo konstruktorius
IntArray::IntArray(const IntArray& other) {
    // TODO: Implementuoti gilųjį kopijavimą (deep copy)
    // 1. Nukopijuoti size
    // 2. Sukurti naują atmintį (new)
    // 3. Nukopijuoti duomenis (ciklas arba memcpy)
    // 4. Logging su [COPY]
}
```

Testavimas:

main.cpp:

```
#include <iostream>
#include "IntArray.h"

int main() {
    std::cout << "=== Gilusis kopijavimas su kopijavimo konstruktoriumi ===" <<
```

```
std::endl;

    IntArray arr1(5);
    arr1.set(0, 10);
    arr1.set(1, 20);
    arr1.set(2, 30);

    std::cout << "arr1: ";
    arr1.print();

    IntArray arr2 = arr1;  // ← Kopijavimo konstruktorius!

    std::cout << "arr2: ";
    arr2.print();

    arr2.set(0, 999);

    std::cout << "arr2 (po keitimo): ";
    arr2.print();

    std::cout << "arr1 (neturėtų pasikeisti): ";
    arr1.print();  // ← arr1 NEPASIKEITĖ!

    return 0;
}
```

Galima išvestis:

```
=== Gilusis kopijavimas su kopijavimo konstruktoriumi ===
[CTOR] IntArray created (size=5, data=0x...)
arr1: IntArray[5]: 10 20 30 0 0
[COPY] IntArray COPIED (size=5, data=0x...)  ← Skirtingas adresas!
arr2: IntArray[5]: 10 20 30 0 0
arr2 (po keitimo): IntArray[5]: 999 20 30 0 0
arr1 (neturėtų pasikeisti): IntArray[5]: 10 20 30 0 0  ← ☑ Teisingai!
[DTOR] IntArray destroyed (size=5, data=0x...)
[DTOR] IntArray destroyed (size=5, data=0x...)
```

Svarbu!

Kopijavimo konstruktoriaus (copy constructor) sintaksė:

```
ClassName(const ClassName& other);
```

Kada kviečiamas:

- IntArray arr2 = arr1; — inicializacija

- IntArray arr3(arr1); — inicializacija
- IntArray arr4{arr1}; — universali inicializacija (C++11)
- func(arr1); — perdavimas reikšme
- return arr1; — grąžinimas reikšme
- vector.push_back(arr1); — konteineriai

Giliojo kopijavimo (deep copy) algoritmas:

1. Kopijuoti primityvius narius, pvz. `size`
2. Sukurti NAUJĄ atmintį (`new`)
3. Nukopijuoti DUOMENIS (ciklas arba `memcpy`)

Failų struktūra:

```
U4/02/
├── IntArray.h
├── IntArray.cpp    ← Su kopijavimo konstruktoriumi
├── main.cpp
└── Makefile
```

---

3 žingsnis: Trijų Taisyklė - Kopijavimo priskyrimas

Direktorija: `U4/03/`

Motyvacija:

Kopijavimo konstruktorius veikia **inicializuojant**. Bet kas su **priskyrimo** (assignment) operacija po inicializacijos?

```
IntArray arr1(5);
IntArray arr2(10);
arr2 = arr1;  // ← Kopijavimo priskyrimo operatorius! (ne kopijavimo
konstruktorius)
```

Reikalavimai:

1. Pridėkite kopijavimo priskyrimo operatorių

Pridėkite **kopijavimo priskyrimo operatorių** (copy assignment operator) į `IntArray` klasę.

2. Pertvarkykite kopijavimo konstruktorių su narių inicializavimo sąrašu

Išnaudokite progą ir **pertvarkykite** 2 žingsnyje sukurtą kopijavimo konstruktorių — naudokite **narių inicializavimo sąrašą** (member initializer list) vietoj priskyrimų konstruktoriaus kūne.

IntArray.h (papildyti):

```
class IntArray {
    // ... ankstesnis kodas ...
public:
    IntArray(int s);
    ~IntArray();
    IntArray(const IntArray& other);

    // NAUJAS: Kopijavimo priskyrimo operatorius
    IntArray& operator=(const IntArray& other);  // ← Pridėti

    // ... kiti metodai ...
};
```

IntArray.cpp — Kopijavimo konstruktorius (pertvarkykite):

```
// Kopijavimo konstruktorius SU narių inicializavimo sąrašu
IntArray::IntArray(const IntArray& other)
    // : size(?),     // TODO: Inicializuoti size
    //                // TODO: Sukurti naują atmintį
{
```

```
    // TODO: Nukopijuoti duomenis (ciklas)
    // TODO: Logging su [OP =]
}
```

IntArray.cpp — Kopijavimo priskyrimas (naujas):

```
// Kopijavimo priskyrimo operatorius
IntArray& IntArray::operator=(const IntArray& other) {
    // TODO: Implementuoti kopijavimo priskyrimą
    // 1. Savęs priskyrimo patikrinimas (if (this == &other))
    // 2. Išvalymas (cleanup: delete[] seną atmintį)
    // 3. Gilusis kopijavimas (kaip kopijavimo konstruktoriuje)
    // 4. Logging
    // 5. return *this
}
```

Testavimas:

main.cpp:

```
#include <iostream>
#include "IntArray.h"

int main() {
    std::cout << "=== Trijų Taisyklė - Kopijavimo priskyrimas ===" << std::endl;

    IntArray arr1(5);
    arr1.set(0, 10);
    arr1.set(1, 20);
    arr1.set(2, 30);

    IntArray arr2(3);
    arr2.set(0, 100);
    arr2.set(1, 200);

    std::cout << "arr1: ";
    arr1.print();
    std::cout << "arr2: ";
    arr2.print();

    arr2 = arr1;  // ← Kopijavimo priskyrimas!

    std::cout << "arr2 (po priskyrimo): ";
    arr2.print();

    arr2.set(0, 999);

    std::cout << "arr2: ";
    arr2.print();
```

```
    std::cout << "arr1 (neturėtų pasikeisti): ";
    arr1.print();

    return 0;
}
```

Tikėtina išvestis:

```
=== Trijų Taisyklė - Kopijavimo priskyrimas ===
[CTOR] IntArray created (size=5, data=0x1c3e84458e0)
[CTOR] IntArray created (size=3, data=0x1c3e84458a0)
arr1: IntArray[5]: 10 20 30 0 0
arr2: IntArray[3]: 100 200 0
[OP =] IntArray ASSIGNED (size=5, data=0x1c3e8445860)
arr2 (po priskyrimo): IntArray[5]: 10 20 30 0 0
arr2: IntArray[5]: 999 20 30 0 0
arr1 (neturėtų pasikeisti): IntArray[5]: 10 20 30 0 0
[DTOR] IntArray destroyed (size=5, data=0x1c3e8445860)
[DTOR] IntArray destroyed (size=5, data=0x1c3e84458e0)
```

Svarbu!

Kopijavimo priskyrimas (copy assignment) vs Kopijavimo konstruktorius (copy constructor):

Atvejis	Sintaksė	Kviečia
Inicializacija	`Type obj2 = obj1;`	Kopijavimo konstruktorius
Inicializacija	`Type obj2(obj1);`	Kopijavimo konstruktorius
Priskyrimas	`obj2 = obj1;`	Kopijavimo priskyrimo operatorius

Narių inicializavimo sąrašas (Member initializer list):

Šiame žingsnyje pertvarkėme kopijavimo konstruktorių, kad panaudoti narių inicializavimo sąrašą: **Kodėl geriau?**

- Nariai **inicializuojami**, ne **priskiriami** (efektyviau)
- Const nariai ir nuorodos **privalo** būti inicializuoti sąraše
- Bazinės klasės konstruktoriai kviečiami sąraše
- Geresnis stilius (Modern C++)

Sintaksė:

```
ClassName::ClassName(params)
    : member1(value1),
      member2(value2)
{
```

```
    // Kūnas
}
```

Kopijavimo priskyrimo algoritmas:

1. **Savęs priskyrimo patikrinimas** (savęs priskyrimo patikrinimas: `if (this == &other)`)
2. **Išvalymas** (cleanup: išvalyti seną atmintį)
3. **Gilusis kopijavimas** (deep copy: nukopijuoti naują)
4. **return this* (grąžinti nuorodą)

Kodėl `return *this`?

- Leidžia grandinę (chaining): `a = b = c;`

Failų struktūra:

```
U4/03/
├── IntArray.h
├── IntArray.cpp    ← Su kopijavimo priskyrimo operatoriumi
├── main.cpp
└── Makefile
```

DALIS 2: MyString

Tikslas: Taikyti Trijų Taisyklę (Rule of Three) labiau **realiam** *use case* — "savo" eilutės klasei.

Kodėl MyString? C stiliaus eilutės (`char*`) — vienas **dažniausių** atvejų, kur reikia Trijų Taisyklės. Suprasime, kaip veikia `std::string` viduje.

---

4 žingsnis: MyString - Trijų Taisyklė

Direktorija: `U4/04/`

Motyvacija:

IntArray buvo apšilimas. Dabar **realus** use case — **eilutės** (String) klasė su `char*`!

Reikalavimai:

Sukurkite `MyString` klasę su Trijų Taisykle (Rule of Three).

Klasė `MyString`:

`private` atributai:

- `char* data` - rodyklė į C stiliaus eilutę (C-string)
- `size_t length` - eilutės ilgis (be `\0`)

`public` metodai:

- `MyString()` - numatytasis (default) konstruktorius (tuščia eilutė)
- `MyString(const char* s)` - C stiliaus eilutės konstruktorius
- `~MyString()` - destruktorius
- `MyString(const MyString& other)` - kopijavimo konstruktorius
- `MyString& operator=(const MyString& other)` - kopijavimo priskyrimo operatorius
- `const char* c_str() const` - grąžina C stiliaus eilutę
- `size_t size() const` - grąžina ilgį
- `bool empty() const` - ar tuščia?
- `void print() const` - išspausdina eilutę

MyString.cpp *skeleton*:

```
// Numatytasis konstruktorius
MyString::MyString() {
    // TODO: Sukurti tuščią eilutę
    // length = 0;
    // data = new char[1];
    // data[0] = '\0';
}
```

```
// C stiliaus eilutės konstruktorius
MyString::MyString(const char* s) {
    // TODO: nullptr patikrinimas, apskaičiuoti ilgį, sukurti atmintį, kopijuoti
}

// Destruktorius
MyString::~MyString() {
    // TODO: Atlaisvinti atmintį
}

// Kopijavimo konstruktorius
MyString::MyString(const MyString& other) {
    // TODO: Nukopijuoti length, sukurti atmintį, kopijuoti duomenis
}

// Kopijavimo priskyrimas
MyString& MyString::operator=(const MyString& other) {
    // TODO: Savęs patikrinimas, išvalymas, gilusis kopijavimas, return *this
}

// TODO: Getters...
```

Testavimas:

main.cpp:

```
#include <iostream>
#include <vector>
#include "MyString.h"

int main() {
    std::cout << "=== MyString Trijų Taisyklė ===" << std::endl;
    // TODO: pabandykite savo scenarijų vietoj apačioje pateikto
    MyString s1;
    MyString s2("Hello");
    MyString s3("C++");

    std::cout << "s1: \""; s1.print(); std::cout << "\" (size=" << s1.size() <<
")" << std::endl;
    std::cout << "s2: \""; s2.print(); std::cout << "\" (size=" << s2.size() <<
")" << std::endl;

    MyString s4 = s2;  // Kopijavimo konstruktorius
    std::cout << "s4 (kopija s2): \""; s4.print(); std::cout << "\"" << std::endl;

    s1 = s3;  // Kopijavimo priskyrimas
    std::cout << "s1 (priskirta iš s3): \""; s1.print(); std::cout << "\"" <<
std::endl;

    // Vector testas su mūsų klasės objektais
    std::vector<MyString> strings;
```

```
    // TODO: įdėkite į vektorių kelis objektus
    // TODO: cikle atspausdinkite vektoriaus turinį

    // !!!
    // Gilesniam supratimui pasitelkite specialiųjų metodų (K, KK, KPO, D)
logging'ą kaip IntArray atveju
    return 0;
}
```

Svarbu!

C stiliaus eilutės (C-style strings):

```
#include <cstring>

strlen(s);        // Ilgis (be '\0')
strcpy(dest, src); // Kopijuoti
strcmp(s1, s2);    // Palyginti
```

Atmintis eilutei:

```
const char* s = "Hello";  // 5 simboliai
// Reikia: 5 + 1 = 6 baitų ('\0' pabaigai)

char* data = new char[strlen(s) + 1];
strcpy(data, s);
```

nullptr patikrinimas:

```
MyString(const char* s) {
    if (s == nullptr) {
        // Sukurti tuščią
        length = 0;
        data = new char[1];
        data[0] = '\0';
        return;
    }
    // ...
}
```

Failų struktūra:

```
U4/04/
├── MyString.h
├── MyString.cpp    ← Trijų Taisyklė
```

```
  ├── main.cpp
  └── Makefile
```

---

5 žingsnis: MyString - Sujungimas (operator+)

Direktorija: `U4/05/`

Motyvacija:

Operatorių perkrovimas (operator overloading) — C++ galimybė apibrėžti, kaip standartiniai operatoriai (`+`, `==`, `[]`, `<<`) veikia su mūsų klasėmis. Tai daro kodą **intuityvų** ir **natūralų**:

```
MyString s3 = s1 + s2;  // Vietoj: s3 = s1.concat(s2);
```

Pradėsime nuo **operator+** (sujungimas - *concatenation*).

Reikalavimai:

Pridėkite `operator+` į `MyString` klasę.

MyString.h (papildyti):

```
class MyString {
    // ... ankstesnis kodas ...
public:
    // ... ankstesnis kodas ...

    // NAUJAS: Sujungimas
    MyString operator+(const MyString& other) const;
};
```

MyString.cpp (papildyti):

```
// Sujungimas
MyString MyString::operator+(const MyString& other) const {
    // TODO: Implementuoti sujungimą (concatenation)
    // 1. Apskaičiuoti naują ilgį
    // 2. Sukurti naują atmintį
    // 3. Nukopijuoti šio objekto eilutę
    // 4. Prijungti kito objekto eilutę
    // 5. Grąžinti rezultato objektą
}
```

Testavimas:

main.cpp:

```
#include <iostream>
#include "MyString.h"

int main() {
    std::cout << "=== MyString Sujungimas ===" << std::endl;

    MyString s1("Hello");
    MyString s2("C++");

    std::cout << "s1: \""; s1.print(); std::cout << "\"" << std::endl;
    std::cout << "s2: \""; s2.print(); std::cout << "\"" << std::endl;

    MyString s3 = s1 + s2;
    std::cout << "s3 = s1 + s2: \""; s3.print(); std::cout << "\"" << std::endl;

    // Grandinė (chaining)
    MyString s4 = s1 + s2 + MyString("!");
    std::cout << "s4 = s1 + s2 + \"!\": \""; s4.print(); std::cout << "\"" <<
std::endl;

    return 0;
}
```

Svarbu!

Operatoriaus perkrovimas (operator overloading) sintaksė:

```
ReturnType operator+(const Type& other) const;
```

strcat vs strcpy:

```
strcpy(dest, src);   // Kopijuoja src → dest (pradžioj)
strcat(dest, src);   // Prijungia src prie dest (pabaigoj)
```

Failų struktūra:

```
U4/05/
├── MyString.h
├── MyString.cpp    ← Su operator+
├── main.cpp
└── Makefile
```

---

6 žingsnis (BONUS): MyString - Kiti operatoriai

Direktorija: `U4/BONUS/`

Motyvacija:

Išplėsti MyString su **daugiau operatorių**!

Reikalavimai:

Pridėkite **bent du** iš šių operatorių:

1. operator[] - Indeksavimas:

```
// MyString.h:
char operator[](size_t index) const;      // Tik skaitymas
char& operator[](size_t index);            // Skaitymas ir rašymas

// MyString.cpp
char MyString::operator[](size_t index) const {
    // TODO: Ribų patikrinimas + grąžinti data[index]
}

char& MyString::operator[](size_t index) {
    // TODO: Analogiškai (grąžina nuorodą)
}
```

2. operator== - Palyginimas:

```
// MyString.h:
bool operator==(const MyString& other) const;

// MyString.cpp
bool MyString::operator==(const MyString& other) const {
    // TODO: Palyginti length + strcmp
}
```

3. operator<< - Spausdinimas:

```
// MyString.h:
friend std::ostream& operator<<(std::ostream& os, const MyString& s);

// MyString.cpp
std::ostream& operator<<(std::ostream& os, const MyString& s) {
    // TODO: Išspausdinti s.data į os stream
}
```

4. operator< - Rikiavimas/rūšiavimas:

```
// MyString.h:
bool operator<(const MyString& other) const;

// MyString.cpp
bool MyString::operator<(const MyString& other) const {
    // TODO: Leksikografinis palyginimas (strcmp)
}
```

Testavimas:

Testuokite savo implementuotus operatorius!

Svarbu!

Draugiškos funkcijos (`friend` functions):

- `operator<<` turi būti draugas (`friend`) — kairėje `ostream`, ne MyString

Const vs non-const `operator[]`:

```
const MyString s("Hello");
cout << s[0];  // ← const versija
// s[0] = 'h';  // ← Klaida! (const objektas)

MyString s2("C++");
s2[0] = 'c';   // ← non-const versija
```

Failų struktūra:

```
U4/BONUS/
├── MyString.h     ← Su operatoriais
├── MyString.cpp   ← Su operatoriais
├── main.cpp       ← Jūsų testai
└── Makefile
```

GitLab direktorijų struktūra

```
└── U4/
    ├── README.md          ← Užduoties santrauka (PRIVALOMA)
    ├── 01/                ← IntArray problema
    ├── 02/                ← IntArray kopijavimo konstruktorius
    ├── 03/                ← IntArray Trijų Taisyklė
    ├── 04/                ← MyString Trijų Taisyklė
    ├── 05/                ← MyString sujungimas
    └── BONUS/             ← MyString kiti operatoriai (optional)
```

Moodle pateikimas

Kaip įprasta

README.md šablonas:

```
# U4: Trijų Taisyklė ir Operatorių Perkrovimas

**Studentas:** Vardas Pavardė
**Grupė:** X

## Žingsnių santrauka

| Žingsnis | Direktorija | Aprašymas |
|----------|-------------|-----------|
| 1 | `01/` | IntArray paviršinio kopijavimo problema |
| 2 | `02/` | IntArray kopijavimo konstruktorius |
| 3 | `03/` | IntArray Trijų Taisyklė |
| 4 | `04/` | MyString Trijų Taisyklė |
| 5 | `05/` | MyString sujungimas (operator+) |
| BONUS | `BONUS/` | MyString kiti operatoriai (optional) |

## BONUS operatoriai (jei darėte)

- [ ] operator[] (indeksavimas)
- [ ] operator== (palyginimas)
- [ ] operator<< (spausdinimas)
- [ ] operator< (rikiavimas)

## Kompiliavimas

```bash
```
```

Pastabos

[Jūsų pastabos, sunkumai, įžvalgos...]